Informe de la práctica 10

Autenticación JWT en PharmaMobil

Estudiante: Grease Duran Castro

Fecha: 13 de octubre de 2026

Proyectos: PharmaMobil y PharmaSoft

Rama de trabajo: feature/auth-jwt-duran

Repositorio: https://github.com/GreysiLL/AP10

1 Objetivo y resultado

Implementamos el inicio de sesión de PharmaMobil contra PharmaSoft mediante JWT. La aplicación guarda el token de forma cifrada, lo adjunta automáticamente a las peticiones y muestra las acciones que corresponden a cada usuario. En Android comprobamos el login, la recuperación de sesión, los permisos, las respuestas 401 y 403 y el CRUD autenticado de productos.

El trabajo se realizó sobre el proyecto existente. Se mantuvo Oracle en Docker y se incorporó la seguridad del backend del curso. El código para el almacenamiento de iOS está preparado, pero su compilación y ejecución quedan pendientes porque no se dispone de una computadora Mac con Xcode.

2 Herramientas utilizadas

Herramienta	Uso en la práctica
Android Studio y emulador Pixel 9	Compilación y ejecución de PharmaMobil en Android 16 API 36
Docker y Oracle Free	Base de datos del backend en localhost puerto 1522
Spring Boot y Maven	Servicios REST y autenticación de PharmaSoft
Kotlin Multiplatform y Ktor	Código compartido y consumo de la API
Postman y Newman	Pruebas de endpoints y permisos
Git y GitHub	Control de versiones y entrega del proyecto

3 Usuarios de prueba

Usuario	Contraseña	Permisos principales
admin	Admin123	Administración y acceso completo
vendedor	Vendedor123	Lectura del catálogo, clientes y ventas
almacen	Almacen123	Lectura y escritura del catálogo

4 Implementación realizada

Backend y base de datos

Actualizamos PharmaSoft a partir del commit de seguridad fff44f99c06d4ca0e71809b591c64435f25978d2 del repositorio del profesor. Conservamos la configuración local de Oracle y la migración V1. Flyway aplicó las migraciones V2 y V3 para usuarios, roles, permisos y cuentas de prueba. POST /api/v1/auth/login valida las credenciales y devuelve el JWT; GET /api/v1/auth/me permite consultar la sesión actual.

Dominio y repositorio de autenticación

Se agregaron Credenciales, Sesion, ErrorSesion y AuthRepository al dominio, junto con los casos de uso de inicio y cierre de sesión. AuthRepositorioRest envía el usuario y la contraseña al backend, transforma la respuesta al modelo de sesión y guarda el token. Después del login y del cierre de sesión se limpia la caché del proveedor bearer para que Ktor use el token correspondiente.

Almacenamiento y cliente HTTP

AlmacenSeguroAndroid utiliza EncryptedSharedPreferences con una MasterKey protegida por Android Keystore, según la guía. Se deshabilitaron las copias de seguridad de la aplicación. AlmacenSeguroIos utiliza Keychain y requiere validación en una Mac. El plugin Auth de Ktor envía el bearer automáticamente al backend autorizado, exceptuando el login. Logging utiliza HEADERS en depuración y sanea Authorization; el registro muestra *** en lugar del token.

Interfaz y manejo de errores

LoginScreen valida los campos y bloquea el envío durante la carga. Al abrir la aplicación se consulta la sesión almacenada y se valida mediante auth/me. Un 401 en una operación protegida borra la sesión y vuelve al login con un aviso; un 403 muestra falta de permiso y conserva la sesión. La interfaz oculta las acciones del catálogo y la sección Clientes según los permisos. También se conectó Clientes al repositorio REST.

Diseño y control de versiones

Al finalizar se mejoró el tema, la tipografía, el login y las tarjetas de productos y clientes. La lista de inventario queda visible y el formulario se abre para crear o editar. Este cambio de diseño quedó registrado en el commit e488658 y compiló correctamente. Las capturas de este informe corresponden a las pruebas funcionales anteriores al cambio visual y se conservan sin modificaciones.

5 Resultados de verificación

Comprobación	Resultado registrado
Pruebas automatizadas de la aplicación	77 aprobadas, 0 fallos y 0 errores
Colección Postman ejecutada con Newman	50 solicitudes enviadas y 68 comprobaciones aprobadas
Opciones no ejecutadas en Newman	3 solicitudes opcionales de venta y creación de usuario
CRUD de productos desde Android	POST 201, GET 200, PUT 200 y DELETE 204
Token inválido en recurso protegido	401, limpieza de sesión y regreso al login
Acceso sin permiso con almacén	403 y sesión conservada
Compilación tras el cambio de diseño	BUILD SUCCESSFUL
iOS	Código preparado, ejecución pendiente

CRUD autenticado

Creamos el producto exclusivo de prueba PruebaS10-144402, con precio 2.50 y stock 2. Se consultó y actualizó su precio a 3.75; posteriormente se eliminó de forma lógica. El registro final conserva el producto inactivo y el inventario activo volvió a tres productos. Las capturas y ktor-android.log registran estas operaciones.

Alcance de las pruebas de permisos

La matriz inicial del backend incluye 23 comprobaciones. Para varias escrituras se envió un cuerpo vacío: 400 indica que el usuario pasó la autorización y llegó a la validación; 403 indica que carece del permiso. Estas respuestas no acreditan una venta registrada. El 401 de la aplicación se provocó con un token deliberadamente inválido, sin esperar una hora de expiración real.

Colección para el profesor

La carpeta postman contiene la colección completa, el entorno local, el esquema OpenAPI y las instrucciones. La colección incluye 53 solicitudes y contempla 28 operaciones de la API. Los tokens se obtienen mediante los logins y no se incluyen en los archivos exportados. Las pruebas CRUD generan sus propios datos temporales; ventas y creación de usuarios son opcionales y permanecen desactivadas por defecto.

6 Evidencias de ejecución

Las páginas siguientes reúnen las 19 capturas existentes. Cada figura indica la acción realizada y su resultado. Los archivos originales y los registros complementarios se encuentran en docs/SESION10 dentro del proyecto.

Login y validación

Se verificó que el login solicita usuario y contraseña y que una respuesta 401 del login muestra un error de credenciales.

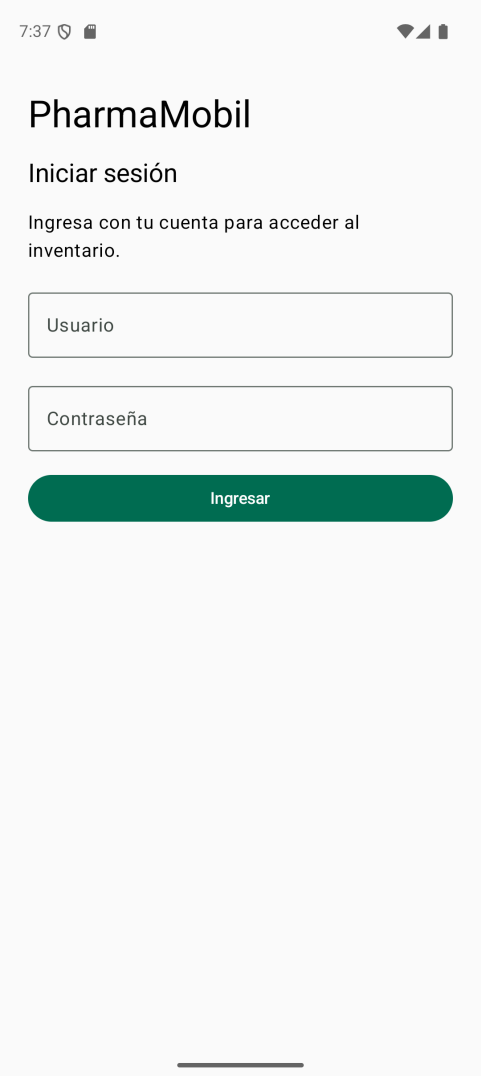


Figura 1. Pantalla de inicio de sesión sin una sesión activa.

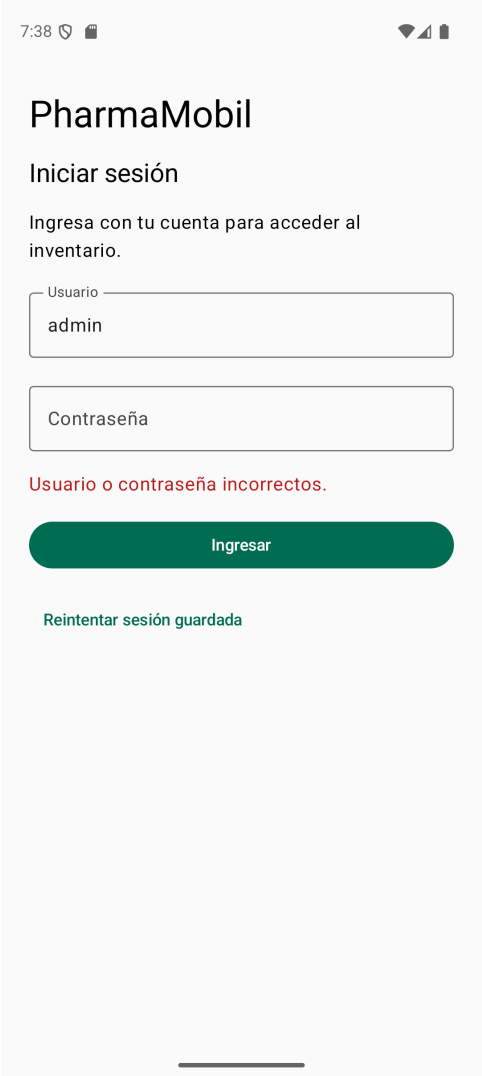


Figura 2. Credenciales incorrectas informadas al usuario.

Sesión autenticada y recuperación

El token persiste en el almacén seguro. Al reabrir, la aplicación consulta auth/me y recupera el inventario sin solicitar nuevamente la contraseña.

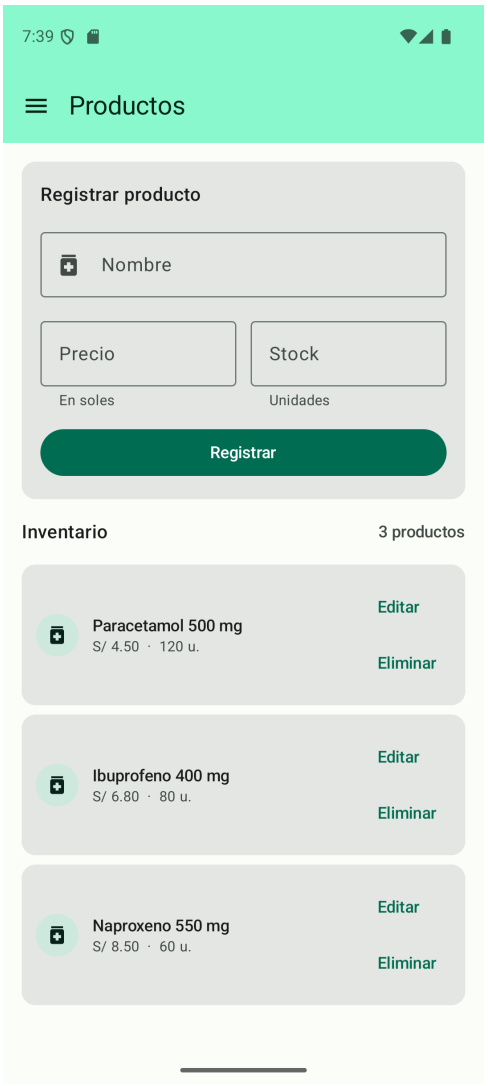


Figura 3. Inventario disponible para admin con acciones de escritura.

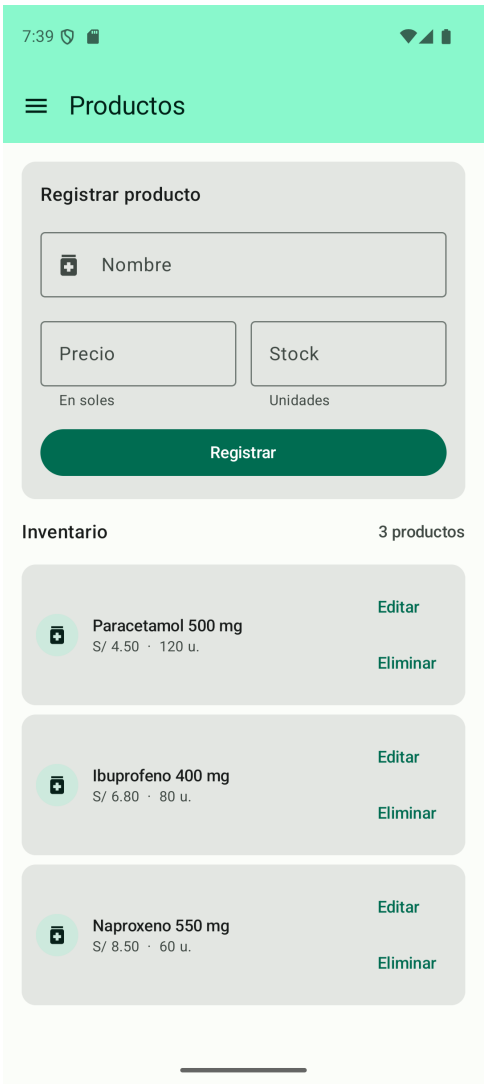


Figura 4. Sesión recuperada después de cerrar completamente y reabrir la aplicación.

Cierre de sesión y rol vendedor

Se comprobó la limpieza de sesión y la presentación del catálogo según los permisos del vendedor.

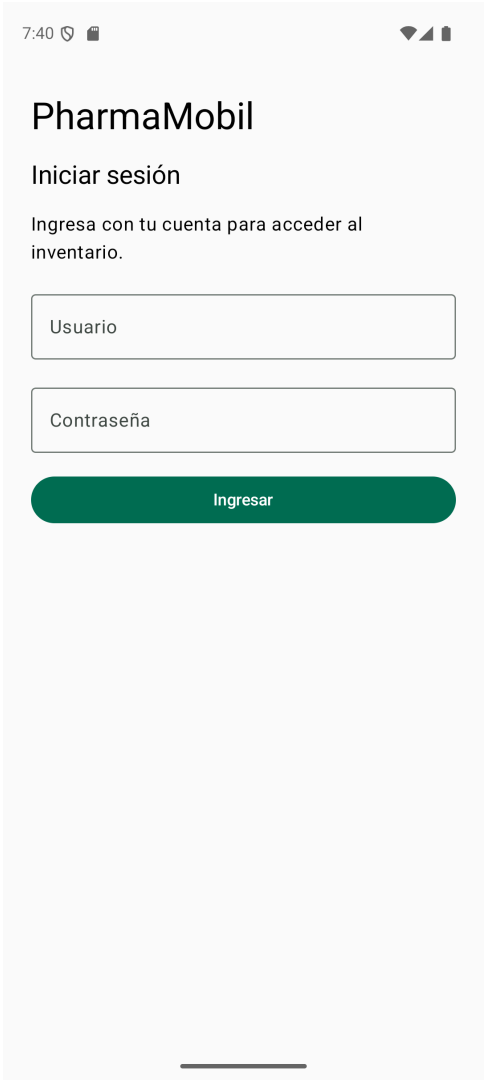


Figura 5. Cerrar sesión devuelve a la pantalla de login.

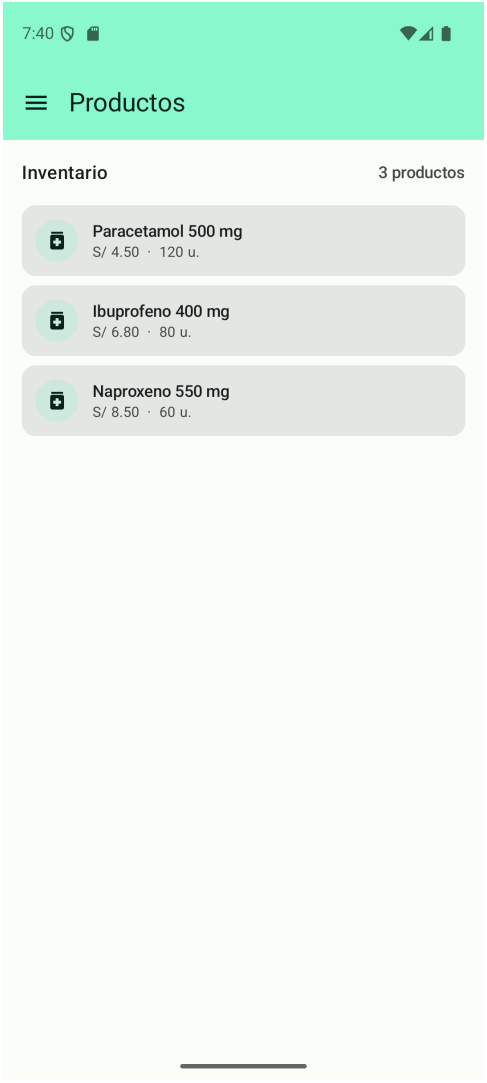


Figura 6. Vendedor puede leer el catálogo; las acciones de escritura se ocultan.

Permisos de vendedor y almacén

Los permisos recibidos desde el backend controlan las opciones disponibles en la aplicación.

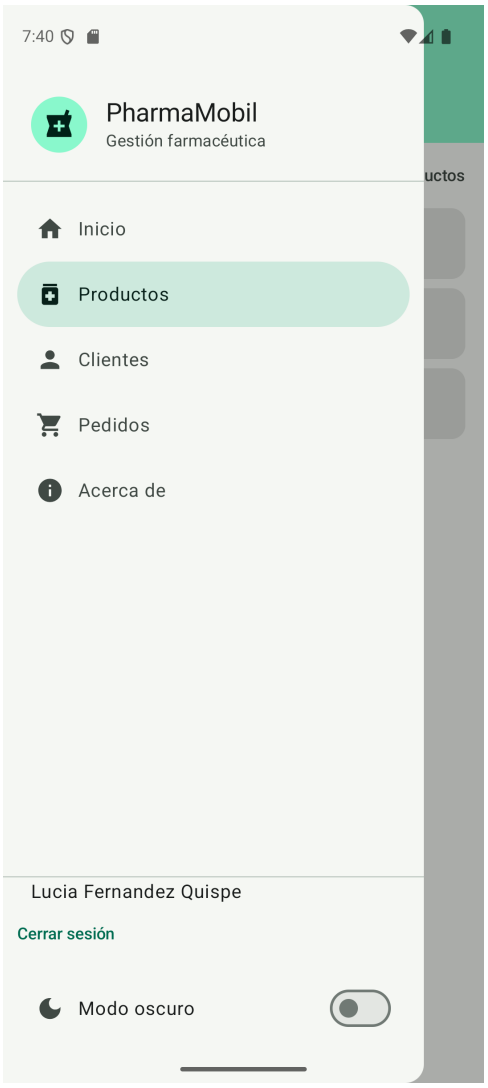


Figura 7. El menú del vendedor permite acceder a Clientes.

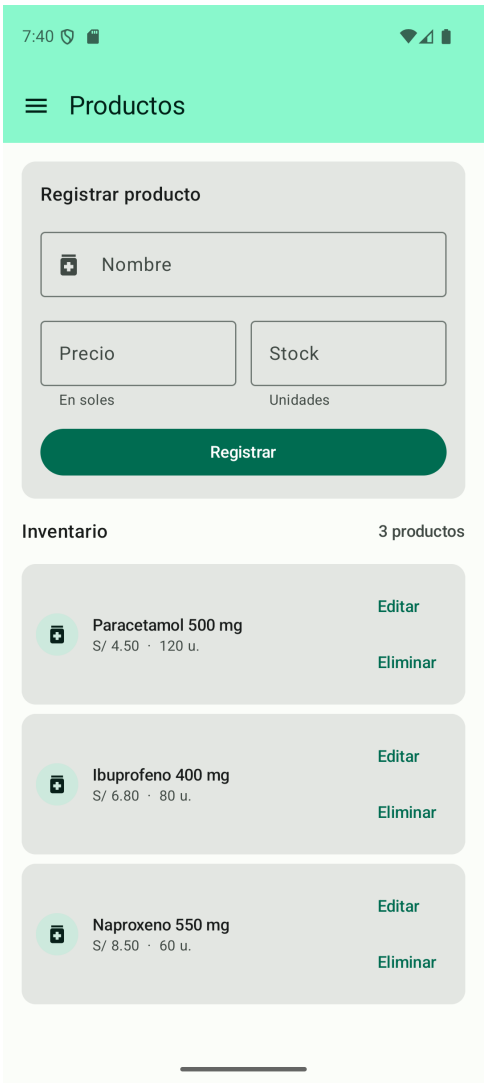


Figura 8. El usuario almacén tiene acciones de escritura del catálogo.

Menú de almacén y respuesta 403

La pantalla de comprobación de seguridad existe solamente en la versión debug. Permite verificar deliberadamente el acceso a un recurso no autorizado.

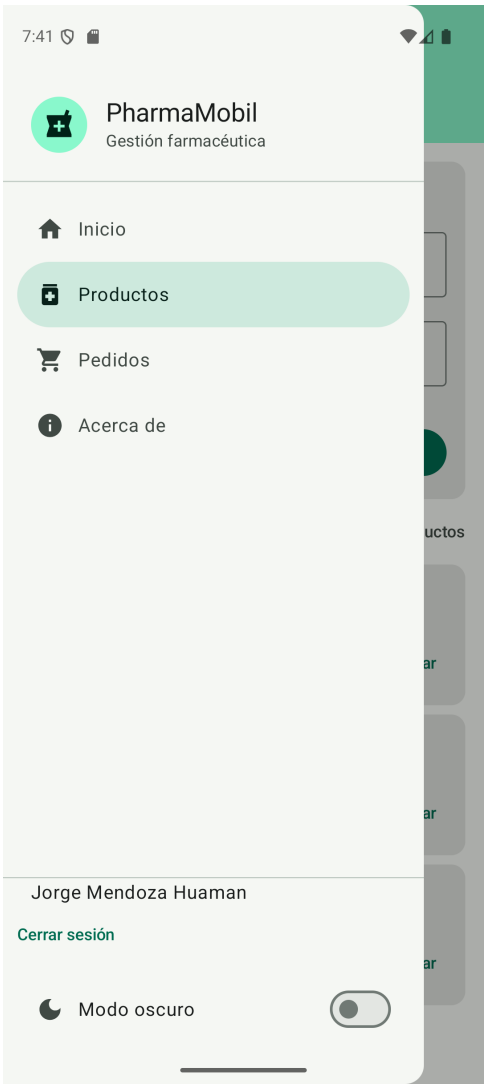


Figura 9. La sección Clientes no se ofrece al usuario almacén.

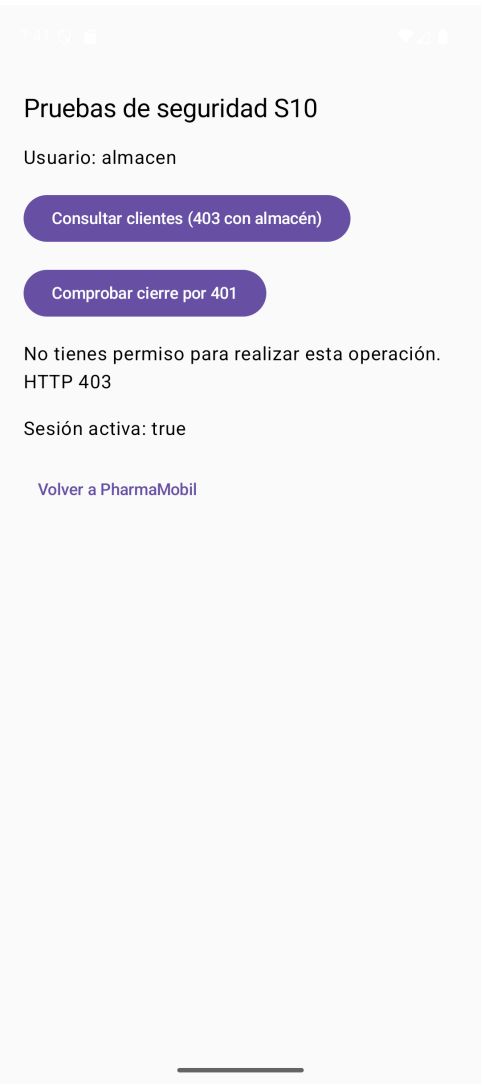


Figura 10. GET clientes provoca 403 y mantiene la sesión activa.

Respuesta 401 y retorno al login

El flujo de un 401 protegido se distingue del error de credenciales del login y del 403 por falta de permiso.

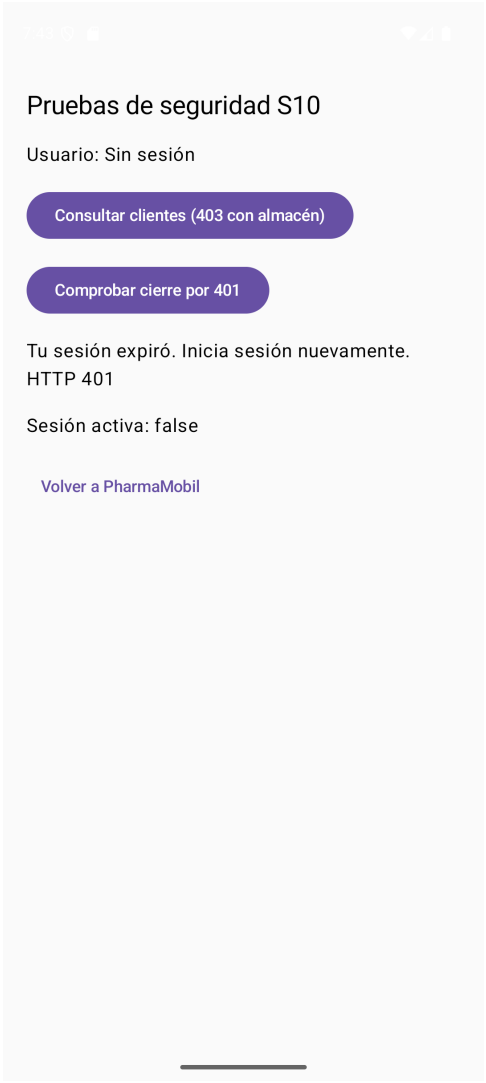


Figura 11. Un token inválido provoca 401 y limpieza de la sesión.

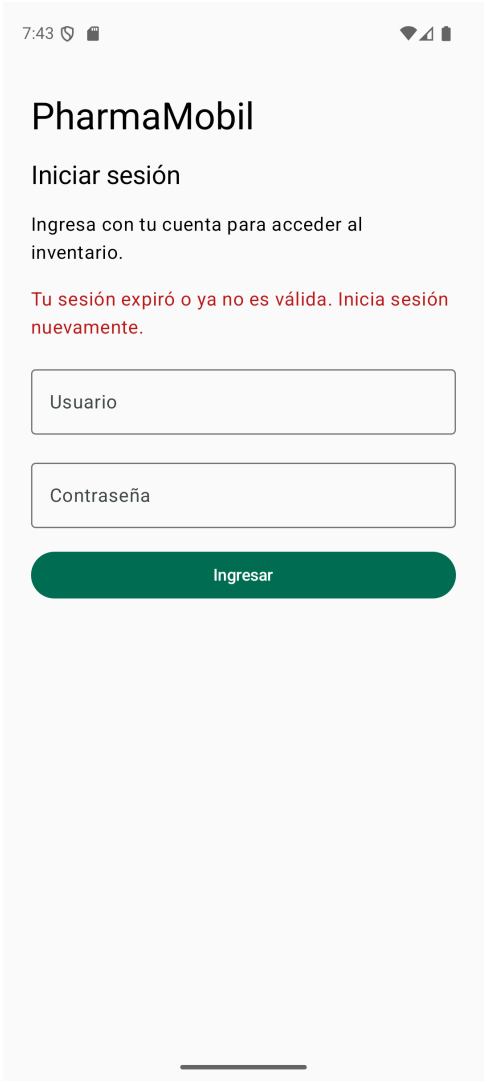


Figura 12. La aplicación vuelve al login con el aviso de sesión expirada.

Registro y actualización de producto

Se ejecutaron las operaciones POST, GET por ID y PUT utilizando el token que adjunta automáticamente Ktor.



Figura 13. Producto de prueba registrado desde Android con autenticación.



Figura 14. Producto consultado y actualizado a precio 3.75.

Eliminación de producto

DELETE devolvió 204. El producto quedó inactivo y el listado de productos activos volvió a su estado anterior a la prueba.

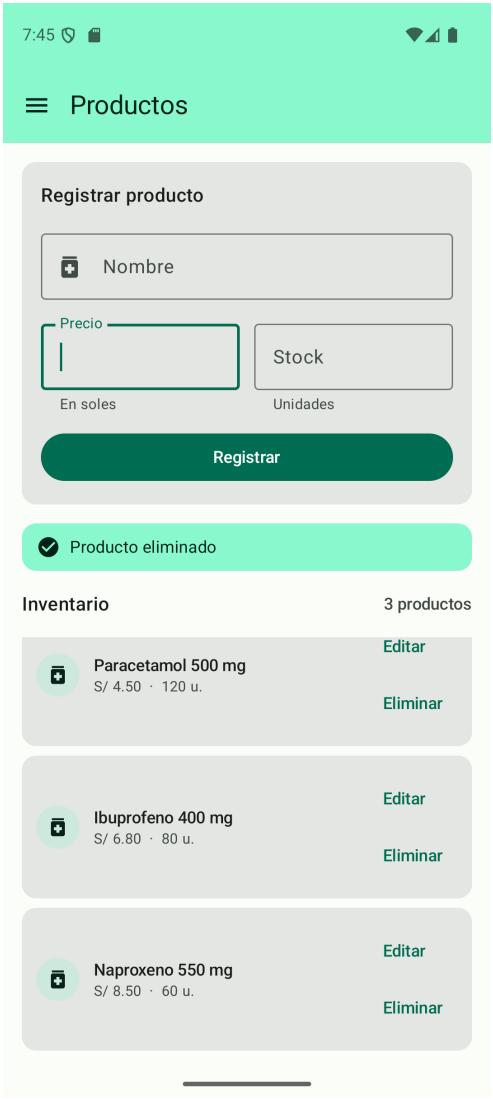


Figura 15. Producto de prueba eliminado lógicamente del inventario activo.

Comprobaciones adicionales en Swagger

Swagger permite contrastar una petición sin token, una permitida y una rechazada por permisos. Las capturas no muestran el token del ejemplo curl.

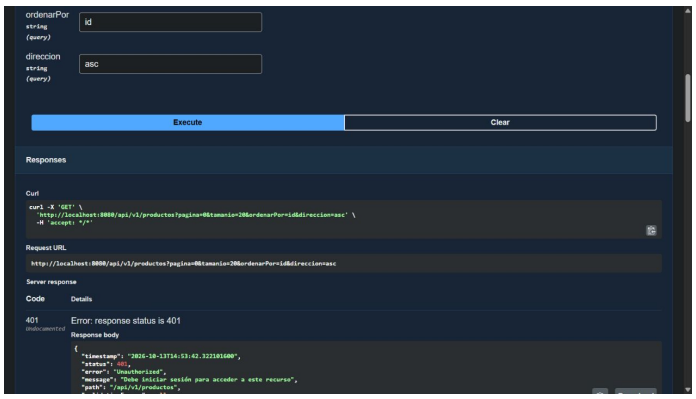


Figura 16. GET productos sin token responde 401.

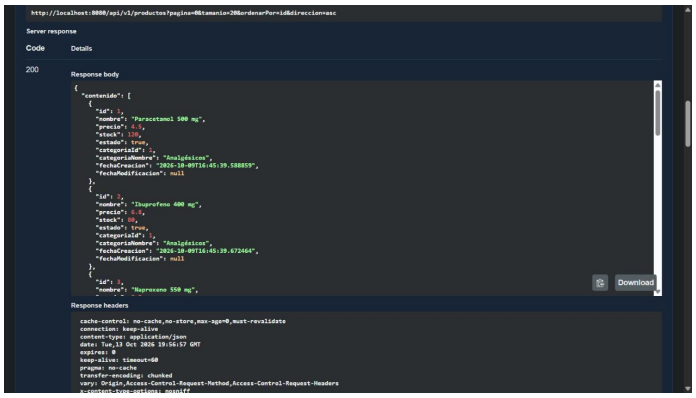


Figura 17. GET productos con admin autorizado responde 200.

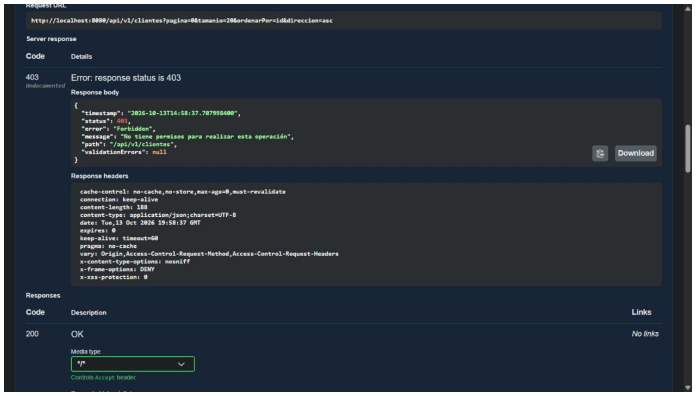


Figura 18. GET clientes con almacén autorizado responde 403.

Registro de Authorization

El registro sanea la cabecera de autenticación y permite comprobar la petición sin divulgar el JWT. La captura procede de la ejecución real compartida desde Android Studio.

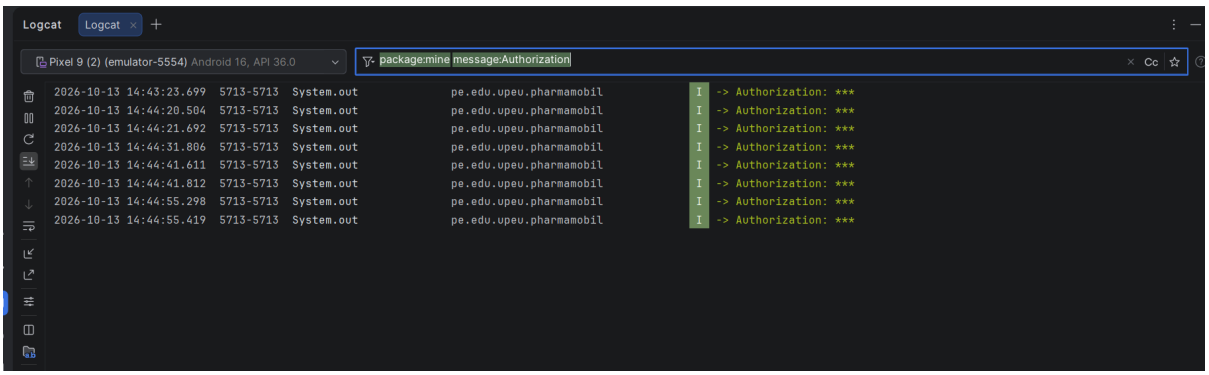


Figura 19. Logcat muestra Authorization: *** en las peticiones de PharmaMobil.

Archivos de respaldo de resultados

docs/SESION10/pruebas.json resume las 77 pruebas; bitacora-android.json registra la ejecución en el emulador; matriz-seguridad.json contiene las respuestas por rol; almacen-seguro-verificacion.json documenta la revisión del almacenamiento cifrado. postman/verificacion-postman.json y ejecucion-newman.log respaldan las 68 comprobaciones de la colección.

7 Cómo comprobar el proyecto

1. Descargar la rama feature/auth-jwt-duran de https://github.com/GreysiLL/AP10 y abrir PharmaMobil en Android Studio. Sincronizar Gradle y seleccionar androidApp.

2. Preparar el backend PharmaSoft con su configuración de seguridad y Oracle disponible. En este entorno, compose.oracle.yaml inicia Oracle Free en el puerto 1522 y PharmaSoft publica la API en http://localhost:8080. La app Android utiliza http://10.0.2.2:8080 para acceder desde el emulador.

3. Iniciar el backend y ejecutar androidApp. Probar los tres usuarios de la tabla: admin permite administrar; vendedor puede leer productos y trabajar con clientes; almacén puede administrar catálogo y no debe tener la sección Clientes.

4. Importar postman/PharmaSoft_S10_Completa.postman_collection.json y postman/PharmaSoft_S10_Local.postman_environment.json en Postman. Seleccionar el entorno local y ejecutar la colección desde el inicio para obtener los tokens. Revisar el README de la carpeta antes de activar las opciones de ventas o creación de usuarios.

5. Ejecutar verificar-sesion10.ps1 desde PowerShell para compilar el APK y correr las pruebas del proyecto. Mantener el backend y Docker encendidos para las comprobaciones manuales y de Postman.

8 Conclusiones

La aplicación quedó integrada con la autenticación JWT de PharmaSoft. La sesión se guarda de forma cifrada en Android, se recupera al reabrir y se elimina al cerrar sesión o recibir un 401 protegido. Los permisos del backend se reflejan en la interfaz y el 403 conserva la sesión. Las pruebas de Android, el CRUD real y la colección de Postman respaldan estos resultados.

La validación de iOS continúa pendiente y debe realizarse en una Mac con Xcode antes de afirmar que el proyecto funciona autenticado en ambas plataformas. Las evidencias incluidas corresponden a Android y al backend; el cambio visual posterior fue compilado y las capturas originales se conservaron.

9 Fuentes y archivos del proyecto

Guía práctica 10 Autenticación JWT, entregada por el docente, especialmente las secciones 8 y 9 de entregables y evaluación.

Backend del curso: https://github.com/dreyna/pharmaSoft, commit de seguridad fff44f99c06d4ca0e71809b591c64435f25978d2.

Proyecto y evidencias: https://github.com/GreysiLL/AP10/tree/feature/auth-jwt-duran. Carpeta docs/SESION10 y carpeta postman.